

02_health-py

#python #api

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ app/  
        └─ api/  
            └─ routes/  
                └─ health.py
```

El archivo `health.py` se encuentra dentro de la carpeta de rutas del backend:

```
backend/app/api/routes/
```

Este archivo define el endpoint de comprobación de estado de la API.

Su función principal es permitir verificar rápidamente si el backend está funcionando y si puede comunicarse correctamente con la base de datos PostgreSQL.

Este router se importa y se registra en `backend/app/main.py` mediante estas líneas:

```
from app.api.routes.health import router as health_router
```

```
app.include_router(health_router)
```

Gracias a esto, el endpoint definido en `health.py` queda incorporado a la aplicación FastAPI principal.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,220p' backend/app/api/routes/health.py
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,220p'
```

Indica que se impriman las líneas de la 1 a la 220.

```
backend/app/api/routes/health.py
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
from __future__ import annotations

from fastapi import APIRouter, Depends
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.db.session import get_db

router = APIRouter(prefix="/health", tags=["health"])

@router.get("")
def health(db: Session = Depends(get_db)):
    db.execute(text("SELECT 1"))
    return {"status": "ok", "db": "ok"}
```

4 Función general del archivo

El archivo `health.py` define un endpoint de salud para comprobar el estado básico del backend.

El endpoint permite verificar dos cosas:

1. Que la API FastAPI está arrancada.
2. Que la API puede ejecutar una consulta sencilla contra la base de datos.

La ruta final expuesta por este archivo es:

```
GET /health
```

Cuando se llama a este endpoint, el backend ejecuta una consulta simple:

```
SELECT 1
```

Si la consulta se ejecuta correctamente, devuelve:

```
{
  "status": "ok",
  "db": "ok"
}
```

Esto indica que la API funciona y que la conexión con PostgreSQL también responde.

5 Estructura general del archivo

El archivo puede dividirse en cinco bloques:

```
from __future__ import annotations
```

Importación futura para mejorar el manejo de anotaciones de tipos.

```
from fastapi import APIRouter, Depends
from sqlalchemy import text
from sqlalchemy.orm import Session
```

Importaciones externas de FastAPI y SQLAlchemy.

```
from app.db.session import get_db
```

Importación interna del proyecto para obtener una sesión de base de datos.

```
router = APIRouter(prefix="/health", tags=["health"])
```

Creación del router específico para la ruta /health .

```
@router.get("")
def health(db: Session = Depends(get_db)):
    db.execute(text("SELECT 1"))
    return {"status": "ok", "db": "ok"}
```

Definición del endpoint que comprueba la conexión con la base de datos.

Visualmente:

```
health.py
├─ Importación futura
├─ Importaciones de FastAPI
├─ Importaciones de SQLAlchemy
├─ Importación de sesión de base de datos
├─ Creación del router
└─ Endpoint GET /health
```

6 Análisis línea por línea

Importación futura de anotaciones

```
from __future__ import annotations
```

Esta línea activa un comportamiento futuro de Python relacionado con las anotaciones de tipos.

Desglose:

```
from __future__
```

`__future__` es un módulo especial de Python que permite activar características de versiones futuras del lenguaje.

```
import annotations
```

Activa el comportamiento moderno de las anotaciones de tipos.

Las anotaciones de tipos son expresiones como:

```
db: Session
```

En este archivo aparece aquí:

```
def health(db: Session = Depends(get_db)):
```

Con `from __future__ import annotations`, Python puede tratar ciertas anotaciones de forma más flexible, evitando algunos problemas con referencias adelantadas o evaluaciones demasiado tempranas.

En este archivo no es estrictamente imprescindible, pero es una práctica común en proyectos modernos de Python.

Importación de APIRouter y Depends

```
from fastapi import APIRouter, Depends
```

Esta línea importa dos elementos desde FastAPI:

```
APIRouter
Depends
```

APIRouter

`APIRouter` permite crear un conjunto de rutas separado de la aplicación principal.

En vez de definir todos los endpoints directamente en `main.py`, cada archivo puede crear su propio router.

En este caso, `health.py` crea el router:

```
router = APIRouter(prefix="/health", tags=["health"])
```

Después, `main.py` lo incorpora a la aplicación principal:

```
app.include_router(health_router)
```

Esto permite organizar mejor el backend.

Depends

`Depends` es el sistema de inyección de dependencias de FastAPI.

Permite indicar que una función necesita recibir un recurso externo, como una conexión a base de datos.

En este archivo se usa aquí:

```
def health(db: Session = Depends(get_db)):
```

Esto significa que FastAPI debe ejecutar `get_db` para proporcionar una sesión de base de datos al parámetro `db`.

Importación de text

```
from sqlalchemy import text
```

Esta línea importa la función `text` desde SQLAlchemy.

`text` permite escribir una consulta SQL manual como texto y convertirla en una expresión que SQLAlchemy puede ejecutar.

En este archivo se usa aquí:

```
db.execute(text("SELECT 1"))
```

El objetivo es ejecutar una consulta muy simple contra la base de datos para verificar que responde.

Sin `text`, SQLAlchemy no trataría la cadena `"SELECT 1"` como una expresión SQL ejecutable de forma explícita.

Importación de Session

```
from sqlalchemy.orm import Session
```

Esta línea importa la clase `Session` desde el módulo ORM de SQLAlchemy.

Una `Session` representa una sesión de trabajo con la base de datos.

A través de una sesión se pueden hacer operaciones como:

- Consultar datos.
- Insertar registros.
- Actualizar registros.

- Eliminar registros.
- Ejecutar SQL.

En este archivo se utiliza como anotación de tipo:

```
db: Session
```

Esto indica que el parámetro `db` debe ser una sesión SQLAlchemy.

La anotación ayuda a entender el código y facilita el autocompletado en editores como VS Code.

Importación de `get_db`

```
from app.db.session import get_db
```

Esta línea importa la función `get_db` desde:

```
backend/app/db/session.py
```

La ruta de importación se interpreta así:

```
app      → carpeta backend/app/  
db       → subcarpeta backend/app/db/  
session  → archivo session.py
```

`get_db` es una función interna del proyecto que se encarga de proporcionar una sesión de base de datos.

En FastAPI, este tipo de función suele utilizarse como dependencia.

Su papel es:

1. Abrir una sesión de base de datos.
2. Entregarla al endpoint.
3. Cerrarla correctamente al terminar la petición.

En este archivo se utiliza aquí:

```
db: Session = Depends(get_db)
```

Esto hace que el endpoint `health` reciba automáticamente una sesión conectada a PostgreSQL.

Creación del router

```
router = APIRouter(prefix="/health", tags=["health"])
```

Esta línea crea un router de FastAPI.

Desglose:

```
router
```

Nombre de la variable donde se guarda el router.

Es importante que se llame `router`, porque en `main.py` se importa así:

```
from app.api.routes.health import router as health_router
```

```
=
```

Operador de asignación.

Asigna el resultado de `APIRouter(...)` a la variable `router`.

```
APIRouter(...)
```

Crea una instancia de router.

Parámetro `prefix`

```
prefix="/health"
```

Define el prefijo común para todas las rutas de este router.

Esto significa que todos los endpoints definidos dentro de este archivo empezarán por:

```
/health
```

Más abajo se define:

```
@router.get("")
```

Como el decorador usa una cadena vacía, la ruta final será:

```
/health
```

Si el decorador fuera:

```
@router.get("/status")
```

la ruta final sería:

```
/health/status
```

Parámetro `tags`

```
tags=["health"]
```

Define las etiquetas usadas por la documentación automática de FastAPI.

En Swagger, los endpoints se agrupan por tags.

Aquí se usa:

```
health
```

Esto hace que el endpoint aparezca agrupado bajo la sección `health` en:

```
http://localhost:8000/docs
```

Los corchetes indican que `tags` recibe una lista:

```
["health"]
```

Aunque solo haya una etiqueta, FastAPI espera una lista.

Separación visual antes del endpoint

La línea en blanco entre:

```
router = APIRouter(prefix="/health", tags=["health"])
```

y:

```
@router.get("")
```

no afecta a la ejecución.

Sirve para separar visualmente la configuración del router de la definición del endpoint.

Decorador del endpoint

```
@router.get("")
```

Esta línea es un decorador de FastAPI.

Un decorador en Python empieza por `@` y modifica o registra la función que viene justo debajo.

En este caso:

```
@router.get("")
def health(...):
```

significa que la función `health` responderá a peticiones HTTP `GET`.

Desglose:

```
@
```

Indica que se está usando un decorador.

```
router
```

Es el router creado anteriormente.

```
.get
```

Indica que se va a registrar una ruta para el método HTTP `GET`.

```
("")
```

Define la ruta relativa dentro del router.

Como está vacío y el router tiene prefijo `/health`, la ruta final será:

```
GET /health
```

Por tanto, cuando llegue una petición `GET` a `/health`, FastAPI ejecutará la función `health`.

Definición de la función `health`

```
def health(db: Session = Depends(get_db)):
```

Esta línea define la función que se ejecutará cuando se llame al endpoint `/health`.

Desglose:

```
def
```

Palabra clave de Python para definir una función.

```
health
```

Nombre de la función.

```
(...)
```

Paréntesis donde se definen los parámetros de la función.

```
db: Session = Depends(get_db)
```

Parámetro de entrada gestionado por FastAPI.

Parámetro `db`

```
db
```

Nombre del parámetro que recibirá la sesión de base de datos.

Dentro de la función se usa aquí:

```
db.execute(text("SELECT 1"))
```

Anotación `Session`

```
db: Session
```

Indica que `db` es de tipo `Session`, importado desde SQLAlchemy.

Esto no crea la sesión por sí solo. Solo indica el tipo esperado.

Dependencia `Depends(get_db)`

```
= Depends(get_db)
```

Indica que el valor de `db` se obtiene ejecutando la dependencia `get_db`.

FastAPI interpreta esto así:

```
Antes de ejecutar health(), ejecuta get_db().  
El resultado de get_db() se pasa como parámetro db.  
Al terminar la petición, cierra o libera la sesión según esté definido en get_db().
```

Esto evita tener que abrir y cerrar manualmente la conexión dentro de cada endpoint.

Ejecución de consulta SQL simple

```
db.execute(text("SELECT 1"))
```

Esta línea ejecuta una consulta SQL sencilla contra la base de datos.

Desglose:

```
db
```

Es la sesión SQLAlchemy recibida mediante `Depends(get_db)`.

```
.execute(...)
```


Método de SQLAlchemy para ejecutar una instrucción SQL.

```
text("SELECT 1")
```

Convierte la cadena "SELECT 1" en una expresión SQL ejecutable por SQLAlchemy.

Consulta SELECT 1

```
SELECT 1
```

Es una consulta mínima que no depende de ninguna tabla.

Su función es comprobar que la base de datos responde.

No consulta eventos, reglas ni alertas. Solo pregunta a PostgreSQL si puede ejecutar una instrucción simple.

Si la conexión a la base de datos falla, esta línea generará un error.

Si funciona, el endpoint continúa y devuelve la respuesta.

Respuesta del endpoint

```
return {"status": "ok", "db": "ok"}
```

Esta línea devuelve un diccionario de Python.

FastAPI convierte automáticamente este diccionario en una respuesta JSON.

Desglose:

```
return
```

Devuelve un valor desde la función.

```
{"status": "ok", "db": "ok"}
```

Diccionario con dos claves:

```
status  
db
```

La respuesta final será:

```
{  
  "status": "ok",  
  "db": "ok"  
}
```

Significado:

```
status: ok → la API ha respondido correctamente.  
db: ok → la consulta a la base de datos se ha ejecutado correctamente.
```

Resultado final del archivo

Este archivo expone un endpoint:

```
GET /health
```

Su comportamiento es:

1. FastAPI recibe una petición GET /health.
2. Ejecuta la dependencia get_db.
3. Obtiene una sesión SQLAlchemy.
4. Ejecuta SELECT 1 contra PostgreSQL.
5. Si no hay error, devuelve {"status": "ok", "db": "ok"}.

7 Relación con el flujo técnico del laboratorio

health.py no forma parte del flujo principal de ingesta, reglas y alertas, pero sí forma parte del control técnico del sistema.

Permite comprobar que el backend y la base de datos están disponibles antes de probar funcionalidades más complejas.

La relación técnica sería:

```
Usuario / curl / navegador
    ↓
GET /health
    ↓
FastAPI ejecuta health()
    ↓
Depends(get_db) proporciona sesión SQLAlchemy
    ↓
db.execute(text("SELECT 1"))
    ↓
PostgreSQL responde
    ↓
FastAPI devuelve {"status": "ok", "db": "ok"}
```

Dentro del laboratorio, este endpoint sirve como comprobación rápida de estado.

Antes de probar:

```
POST /ingest
GET /events
GET /alerts
POST /rules
```

conviene comprobar:

```
GET /health
```

Si /health falla, probablemente habrá un problema general de backend o base de datos.

8 Errores típicos o puntos importantes

Error de conexión con la base de datos

Si PostgreSQL no está disponible, esta línea puede fallar:

```
db.execute(text("SELECT 1"))
```

Causas posibles:

- El contenedor siem-db no está levantado.
- La variable DATABASE_URL es incorrecta.
- El servicio db no está en la misma red que api.
- PostgreSQL todavía no está listo.
- Las credenciales son incorrectas.

Error en la dependencia get_db

Si la función `get_db` está mal definida en:

```
backend/app/db/session.py
```

el endpoint no podrá obtener una sesión de base de datos.

Esto afectaría también a otros endpoints que dependan de la base de datos.

Error de importación de `router`

En `main.py`, el router se importa con:

```
from app.api.routes.health import router as health_router
```

Por tanto, este archivo debe definir una variable llamada:

```
router
```

Si se cambiara el nombre de la variable, habría que modificar también el import en `main.py`.

Ruta vacía en el decorador

El endpoint usa:

```
@router.get("")
```

Como el router ya tiene:

```
prefix="/health"
```

la ruta final es:

```
/health
```

Si se cambiara a:

```
@router.get("/")
```

la ruta podría quedar como:

```
/health/
```

con barra final. FastAPI suele manejar redirecciones, pero conviene mantener una convención clara.

Endpoint de salud con comprobación real de base de datos

Este endpoint no solo comprueba que FastAPI responde, sino también que la base de datos responde.

Esto lo hace más útil que un endpoint que solo devolviera:

```
{"status": "ok"}
```

Aquí también se comprueba:

```
{"db": "ok"}
```

9 Comandos útiles relacionados

Comprobar el endpoint desde el host:

```
curl http://localhost:8000/health
```

Respuesta esperada:

```
{"status": "ok", "db": "ok"}
```

Comprobar el endpoint desde navegador:

```
http://localhost:8000/health
```

Ver logs de la API:

```
docker logs siem-api
```

Ver logs en tiempo real:

```
docker logs -f siem-api
```

Comprobar que el contenedor de PostgreSQL está activo:

```
docker ps | grep siem-db
```

Comprobar que el contenedor de la API está activo:

```
docker ps | grep siem-api
```

Entrar en el contenedor de la API:

```
docker exec -it siem-api bash
```

Probar importación del router desde dentro del contenedor:

```
docker exec -it siem-api python -c "from app.api.routes.health import router; print(router)"
```

Probar importación de la aplicación completa:

```
docker exec -it siem-api python -c "from app.main import app; print(app.title)"
```

Ejecutar consulta directa en PostgreSQL desde el contenedor de base de datos:

```
docker exec -it siem-db psql -U siem -d siem -c "SELECT 1;"
```

```
Dataview (for inline query '= Depends(get_db)'): Unrecognized function name 'Depends'
```